
Practical - 5

Aim:

Implementation of a knapsack problem using dynamic programming.

Theory:

The 0/1 Knapsack Problem is solved using dynamic programming by breaking down the decision-making process for each item into two choices: exclude the item or include the item. This approach avoids exponential time complexity by storing the solutions to overlapping subproblems in a table, reducing the runtime to $O(n \times W)$, where n is the number of items and W is the maximum weight capacity.

Input:

- n = number of items
- W = maximum capacity of knapsack
- $\text{weight}[i]$ = weight of item i
- $\text{value}[i]$ = value/profit of item i

Algorithm:

1. Create a DP table $\text{dp}[n+1][W+1]$.
2. Initialize the first row and first column to 0.
3. For each item i from 1 to n :
 - For each capacity w from 1 to W :
 - If $\text{weight}[i-1] \leq w$:
 - Either include the item or exclude it.

- Choose the maximum:
 $dp[i][w] = \max(\text{value}[i-1] + dp[i-1][w-\text{weight}[i-1]], dp[i-1][w])$
- Otherwise, the item cannot be included:
 $dp[i][w] = dp[i-1][w]$

4. The maximum possible profit is $dp[n][W]$.

5. Print $dp[n][W]$.

Program Code

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

int knapsack(int W, vector<int>& weight, vector<int>& value, int n) {

    // DP table

    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    // Build the DP table

    for (int i = 1; i <= n; i++) {

        for (int w = 1; w <= W; w++) {
```

```
// If the current item can fit
if (weight[i - 1] <= w) {
    dp[i][w] = max(
        value[i - 1] + dp[i - 1][w - weight[i - 1]],
        dp[i - 1][w]
    );
}
else {
    // Cannot include the current item
    dp[i][w] = dp[i - 1][w];
}
}

return dp[n][W];
}

int main() {
    int n, W;
```

```
cout << "Enter number of items: ";
```

```
cin >> n;
```

```
vector<int> weight(n);
```

```
vector<int> value(n);
```

```
cout << "Enter weights of items: ";
```

```
for (int i = 0; i < n; i++) {
```

```
    cin >> weight[i];
```

```
}
```

```
cout << "Enter values of items: ";
```

```
for (int i = 0; i < n; i++) {
```

```
    cin >> value[i];
```

```
}
```

```
cout << "Enter maximum capacity of knapsack: ";
```

```
cin >> W;
```

```
int maximumValue = knapsack(W, weight, value, n);
```

```
cout << "Maximum value = " << maximumValue << endl;

cout<<"kolusu uday kiran\n";

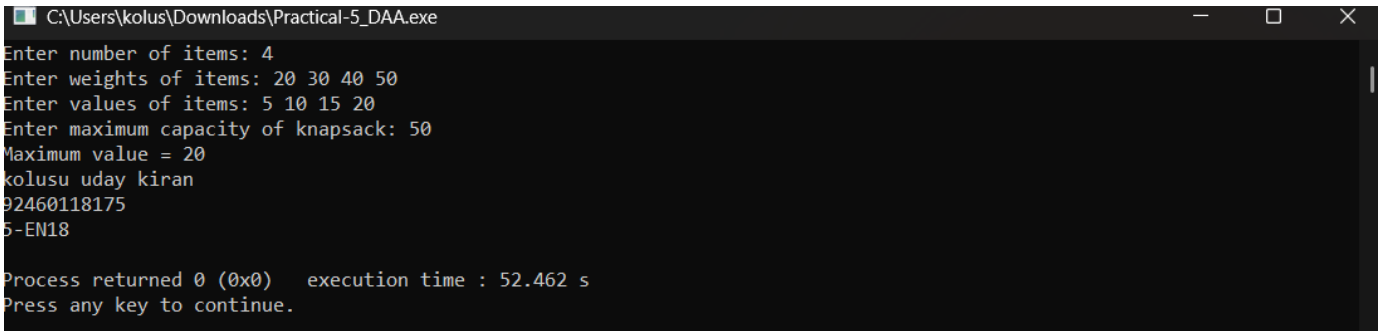
cout<<"92460118175\n";

cout<<"5-EN18\n";

return 0;

}
```

Result :



```
C:\Users\kolus\Downloads\Practical-5_DAA.exe
Enter number of items: 4
Enter weights of items: 20 30 40 50
Enter values of items: 5 10 15 20
Enter maximum capacity of knapsack: 50
Maximum value = 20
kolusu uday kiran
92460118175
5-EN18

Process returned 0 (0x0)   execution time : 52.462 s
Press any key to continue.
```

